



HOCHSCHULE HEILBRONN

Computer & Robot Vision

Documentation

Project Rubik's Cube



Group: Rubik's Cube
Participants: Lukas Gerstlauer, 205293, lgerstla@stud.hs-heilbronn.de
Tim Söns, 204453, tsoens@stud.hs-heilbronn.de
Lecture: Computer & Robot Vision
Lecturer: Prof. Dr. Dieter Maier, Dipl.-Ing. (FH) Joachim Kircher
Date: 31.01.2025
Program: MAS
Faculty: T1

List of contents

1	Introduction	1
2	Technical background	2
2.1	Edge Detection	2
2.1.1	Find contours with OpenCV	2
2.1.2	Canny Filter	2
2.2	Color correction	3
2.3	Color detection	5
3	Program structre	7
3.1	Edge detection	7
3.2	Color correction	9
3.3	Color detection	11
3.4	Merging	12
3.4.1	Grouping.....	13
3.4.2	Assign grid positions.....	14
3.4.3	Combining of dictionaries	14
3.5	Compensation for missing segments.....	14
4	Test and Evaluation	15
5	Prospects	17
6	Bibliography	III
7	List of figures.....	IV
8	List of tables	IV

1 Introduction

In today's digital world, the visual processing of information plays a crucial role. These technologies enable machines to interpret and react to their environment.

The present project was developed as part of the lecture “Computer and Robot Vision” and aims to realize the color recognition of the faces of a Rubik's Cube. The Rubik's Cube poses a number of challenges for color detection, such as different colors of different cubes, similar hues of two colors (red, orange) and different reflection conditions of the colors. Thus, a wide range of different methods and techniques of image processing must be applied to realize a reliable detection under different conditions.



Figure 1: Image of Rubik's Cube under lab conditions

This documentation describes the approach, methods and results of the project. Starting with a theoretical introduction to relevant color models and image processing methods, the development and implementation of the color detection system is then discussed. Finally, the results are evaluated and possible improvements and extensions are discussed.

2 Technical background

The following chapter explains the necessary technical basics of image processing. Edge detection as well as color correction and detection are discussed.

2.1 Edge Detection

Edge detection is a fundamental task in image processing and computer vision, used to identify boundaries within images. It helps in feature extraction and object detection by highlighting significant changes in intensity. Various edge detection techniques exist, but in this project the Canny edge detection algorithm is used.

2.1.1 Find contours with OpenCV

The `cv2.findContours` method in OpenCV is used to detect the boundaries of objects within a binary image. It works by analyzing the edges of connected components, treating white regions as objects and black regions as the background (by default). The function retrieves contours using different retrieval modes (`RETR_TREE`, `RETR_EXTERNAL`, etc.) and approximates them based on the chosen approximation method (`CHAIN_APPROX_SIMPLE`, `CHAIN_APPROX_NONE`). The output consists of a list of contours, where each contour is represented as a sequence of points outlining the object's shape, making it useful for object detection, shape analysis, and image segmentation. [1]

2.1.2 Canny Filter

Edge detection is used to recognise objects based solely on their outline. In order to reliably recognise these edges, John F. Canny developed the so-called Canny filter. This is a multi-stage algorithm for extracting robust and precise edges. [2]

1. Noise reduction using a Gaussian filter

As edge detection is sensitive to image noise, the input image is first smoothed using a 5×5 Gaussian filter

2. Calculation of the intensity gradient

After smoothing, the gradient vector (G_x, G_y) is calculated using Sobel operators. The direction of the gradient is always perpendicular to the edges. It is rounded to one of four angles representing vertical, horizontal and two diagonal directions.

3. Non-maximum suppression

To sharpen edges, the image is checked for local maxima in the gradient direction. If a pixel is not a local maximum, it is set to zero.

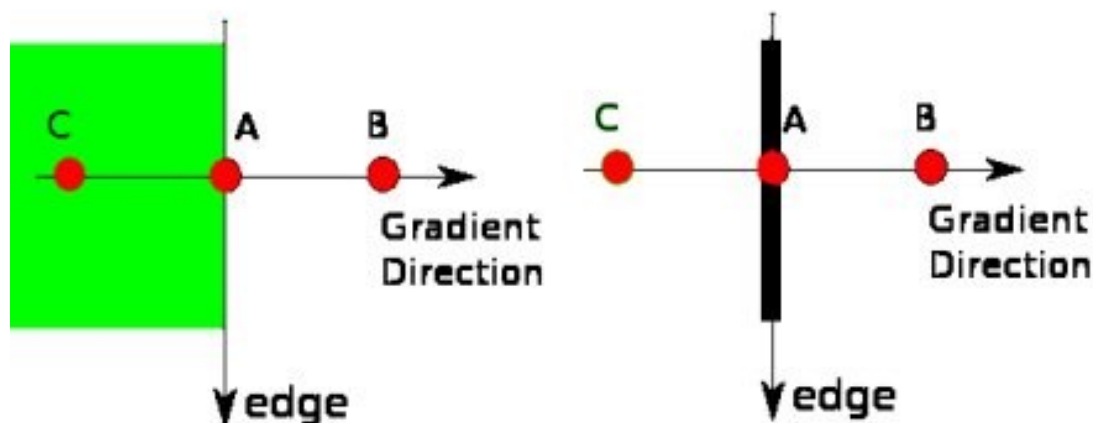


Figure 2: Non-maximum suppression [2]

Point A lies on the edge (in vertical direction). The gradient direction is normal to the edge. Points B and C lie in the gradient direction. Point A is therefore checked with points B and C to see whether it forms a local maximum. If so, it is taken into account for the next step, otherwise it is suppressed (set to zero). The result is a binary image with "thin edges"

4. Hysteresis thresholding

In this phase, real edges are distinguished from unwanted edges. Two threshold values, *minVal* and *maxVal*, are used for this purpose. Edges with a gradient above *maxVal* are considered safe, while those below *minVal* are discarded. Values in between are only accepted as edges if they are connected to safe edges.

As a result, we get strong edges in the image which definitely delimit objects.

2.2 Color correction

As the color detection of the individual areas should also function under different lighting conditions, a so-called color correction is used. This is necessary because the corresponding color areas of the individual colors are defined under neutral, bright light. If the light color now changes, the color values of the surfaces also change accordingly, so that they may slip into other color ranges or even lie outside the defined color ranges. For example, the white surface also appears bluish under blue light. The color correction is intended to remedy this and transform the color back into neutral colors. The correction takes place in the RGB color space [3].

For color correction, an area in the original image that is as uniformly grey as possible is required to determine the ground truth values. The individual color channels should be equally pronounced in the grey area. The ground truth values of the individual color channels can be determined using the histogram of this area. The standard deviations and the position of the maximum values of the histogram are important here:

Standard deviation: $\sigma_r, \sigma_g, \sigma_b$

Position of the maximum value: $\langle r \rangle, \langle g \rangle, \langle b \rangle$



Figure 3: Color correction - Calibrated object [3]

($\sigma_r = 11,7, \sigma_g = 11,1, \sigma_b = 11,2, \langle r \rangle = 174,8, \langle g \rangle = 173,8, \langle b \rangle = 173,2$)

To calibrate a discolored image, the parameters of the same grey area must also be determined. These are the changed standard deviations and the shifted position of the maximum values.

Discolored image: $\sigma'_r, \sigma'_g, \sigma'_b, \langle r' \rangle, \langle g' \rangle, \langle b' \rangle$

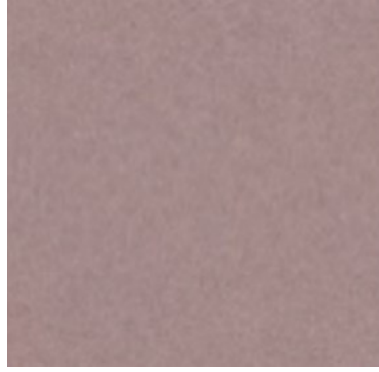


Figure 4: Color correction - Simulated data [1]

$$(\sigma'_r = 9,3, \sigma'_g = 8,9, \sigma'_b = 9,0, \langle r' \rangle = 163,4, \langle g' \rangle = 138,7, \langle b' \rangle = 138,2 \\ c_r = 30; c_g = 0; c_b = 0; c_0 = 0,8)$$

The correction factors can now be calculated from the two sets of values in order to transform the discolored image into the same color and brightness range as the ground truth image. These correction factors are: $\hat{c}_0, \hat{c}_r, \hat{c}_g, \hat{c}_b$

The constant $\hat{c}_0$ is a multiplicative constant and results from the variation of the exposure time or the opening/closing of the aperture.

exposure time or the opening/closing of the aperture. The constants $\hat{c}_r, \hat{c}_g, \hat{c}_b$ are additive constants for the three color channels and result from the colorfulness of external lighting. The constants can be determined using the following formulae:

$$\hat{c}_0 = \frac{1}{3} \left(\frac{\sigma'_r}{\sigma_r} + \frac{\sigma'_g}{\sigma_g} + \frac{\sigma'_b}{\sigma_b} \right) \\ \hat{c}_r = \frac{\langle r' \rangle}{\hat{c}_0} - \langle r \rangle \\ \hat{c}_g = \frac{\langle g' \rangle}{\hat{c}_0} - \langle g \rangle \\ \hat{c}_b = \frac{\langle b' \rangle}{\hat{c}_0} - \langle b \rangle$$

Once the correction constants have been calculated, each pixel of the originally discolored image can be transformed. To do this, the following formula is applied to the three color channels of each pixel in the image:

$$s''(x, y) = \frac{s'(x, y)}{\hat{c}_0} - \hat{c}_i; i = r, g, b$$

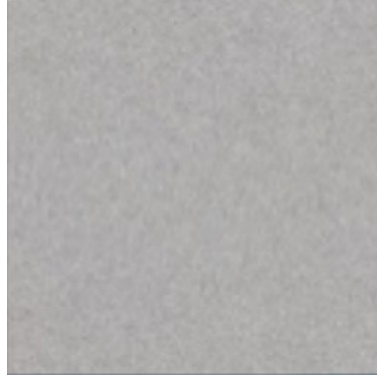


Figure 5: Color correction - re-calibrated image [1]
 $(\sigma'_r = 11,6, \sigma'_g = 11,1, \sigma'_b = 11,2, \langle r' \rangle = 174,4, \langle g' \rangle = 173,2, \langle b' \rangle = 172,8)$

The resulting image therefore has the same color as the original ground truth image.

This technique is only reliable for minor deviations in color and brightness, as it is a linear color correction. Extreme changes in color or brightness lead to non-linear shifts that cannot be compensated for. If a color component changes significantly due to external lighting, important color details may be lost because of the correction. In addition, the image section for determining the parameters in both images must be as identical as possible and a grey area without artefacts to deliver reliable results.

For small changes in color and brightness, however, this is a quick and easy correction option to improve the subsequent color detection.

2.3 Color detection

Color recognition is used to detect areas in the input image that have certain color values. In the example of the Rubik's cube, six different colors can be detected for the six sides of the cube. In order to reliably detect contiguous areas, these should have as homogeneous and uniform a color as possible. The different colors to be recognised should be as different as possible and not be close to each other.

To make color recognition independent of the saturation and brightness of the image and the colors, the HSV color space (Hue, Saturation, Value) is selected.

Three values are also assigned to each pixel in the image to describe the appearance of the pixel. These values are the color hue, the saturation and the brightness.

- **Hue (H):** Describes the pure color and is specified as an angle on a color circle. For example, 0° corresponds to red, 120° to green and 240° to blue in a full 360° circle.
- **Saturation (S):** Indicates how intense a color is. A low saturation leads to a greyer display, while a high saturation makes the color appear as strong as possible.
- **Value (V):** Represents the brightness of a color, where 0 stands for black and 100% for the brightest display level.

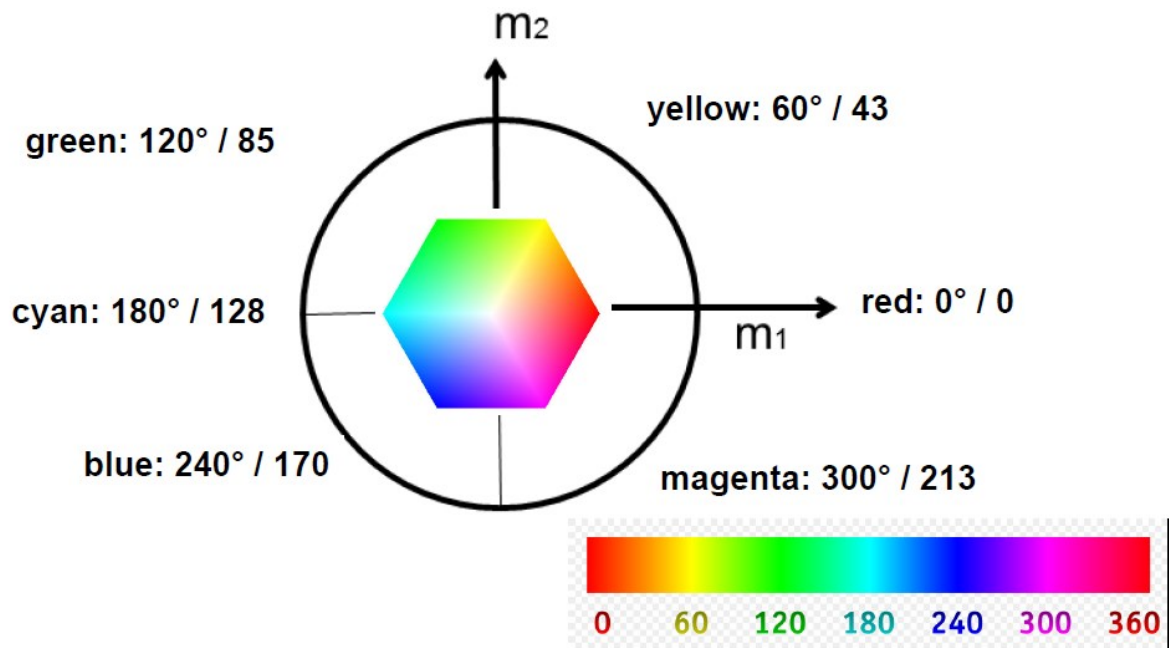


Figure 6: Representation of Hue in HSV color space [3]

A decisive advantage of the HSV color space lies in its robustness against variations in lighting. As the hue (H) is largely independent of the brightness, colors can be reliably detected even under different lighting conditions. This is particularly important for different lighting situations.

After the transformation into the HSV color space, a value range for the H, S and V is defined for each color. If the color values of a pixel lie within this range, the pixel is assigned to the respective color. The value ranges of the individual colors should not overlap, as otherwise double assignments may occur. After assigning each pixel, binary images can be created for each color, which represent the areas that lie within this defined color. Within these binary images, you can search for objects which, in the example of the rubik's cube, represent the faces of one side of the cube. All objects that correspond to the faces are summarised and form the photographed side of the cube.

3 Program structre

The program is divided into different processing steps (see Figure 7: Process) to detect the nine colored segments of the Rubik's cube. First the grey frame in which the cube is located gets detected in the image. Based on the found bounding box the image gets cropped to achieve a region of interest (ROI). This not only delete noise or distracting objects from the image, it also reduces the computational power needed for further processing steps. The gray frame has another important task besides locating the cube. It is the reference object for the following color correction from Chapter 3.2. The color correction is used to get rid of color light or illumination changes which would influence the color detection. Afterwards two different approaches are used to find the 9 segments of the Rubik's cube. Firstly, the cropped frame gets filtered by all colors that are present on the cube. Then a dictionary is created with the found segments. The second approach is an edge-based filter, which detect the segments based on the contours found in the cropped binary image. These findings are also returned as a dictionary of the shown structure Figure 7. After the two methods have detected the segments, it is necessary to sort the found elements and add their grid position to the

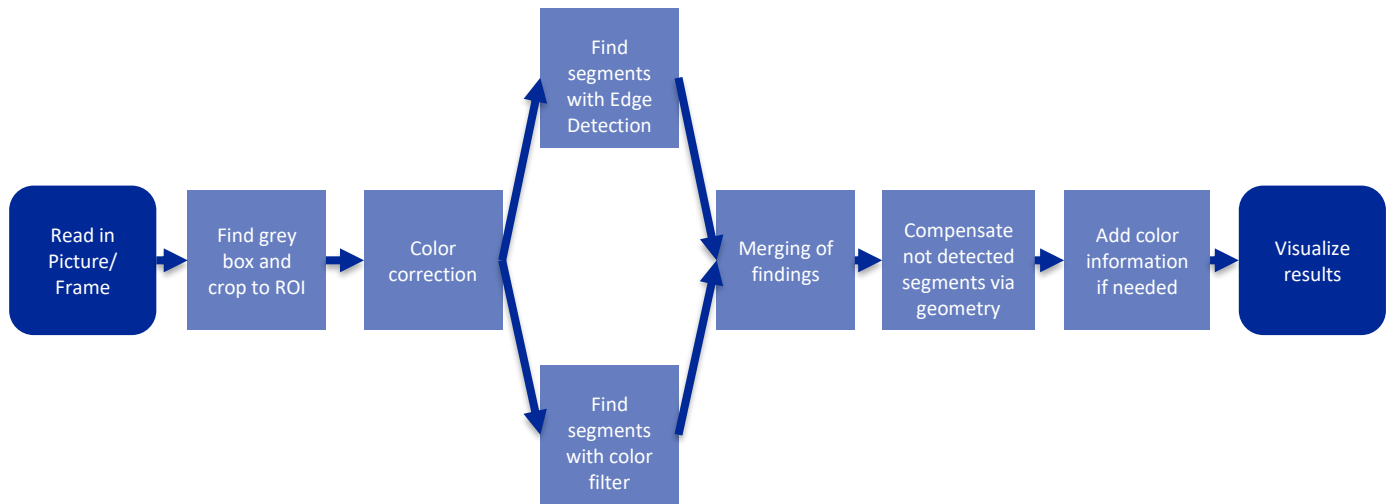


Figure 7: Process Diagram

dictionary entry. Now it is time to merge the two dictionaries and check if all nine segments are present in at least one of them. If there are missing ones a geometric compensation is used to interpolate the missing segments. After all these processing steps all nine segments should be found or compensated. For the ones found via edge detection or by compensation we need to add the color information with another color detection in the centre point of the segment. The following chapters will explain these steps in more detail.

3.1 Edge detection

The `edge_detection` function is designed to find the nine segments of the Rubik's cube based on the edges between the segments. The process begins by searching for black areas in the cropped image. Therefore, the image is first converted to grayscale using OpenCV's `cv2.cvtColor` function. Following this, a binary mask is generated using a threshold operation, where pixels with an intensity below a specified value (40 in this case) are classified as black regions and assigned a value of 255, while all others are set to 0. This thresholding isolates the area which surrounds the single segments of the cube.

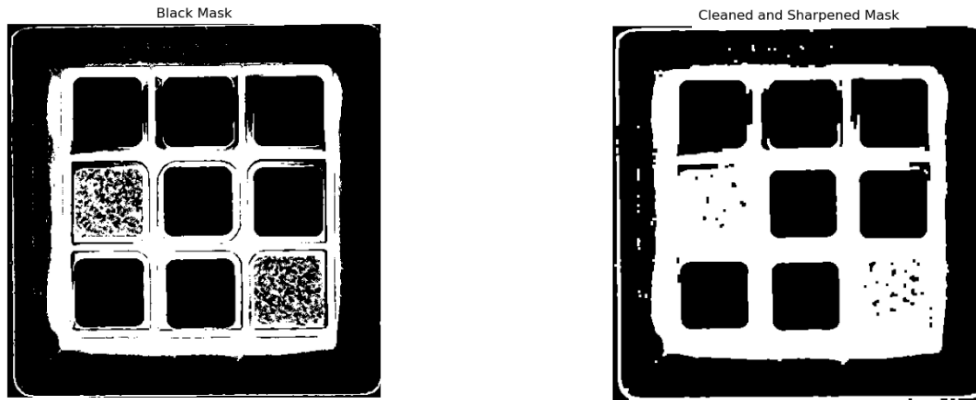


Figure 8: Preprocessing of edge detection

Once the binary mask is created, morphological operations are applied to refine it. These operations serve to remove noise and smooth out irregularities in the mask. Initially, a morphological closing operation is used, which fills small gaps in the black regions and connects disjointed areas. This is achieved with a rectangular kernel of size 3x3, applied iteratively to strengthen the effect. Next, a morphological gradient operation is employed to enhance the edges within the mask. This operation calculates the difference between the dilation and erosion of the mask, effectively highlighting its boundaries. The resulting gradient mask is then combined with the cleaned mask using a bitwise OR operation, further sharpening the edges and preparing the mask for contour detection.

With a refined mask in hand, the function proceeds to locate contours within it. Contours are extracted using OpenCV's `cv2.findContours` function, which identifies and retrieves the edges of all connected regions in the mask. These contours are then analyzed to determine which ones correspond to rectangular shapes. For each detected contour, its perimeter is calculated using `cv2.arcLength`, and this value is used to approximate the contour as a polygon with fewer vertices via the `cv2.approxPolyDP` function. This polygonal approximation ensures that irregular shapes are smoothed out, making it easier to identify rectangles.

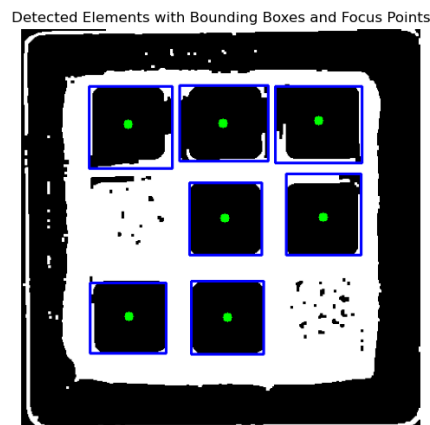


Figure 9: Result of edge detection

After approximating the contours as polygons, a filtering step is applied to exclude shapes that do not meet certain area criteria. The function calculates the area of each polygon using `cv2.contourArea` and retains only those with areas within a specific range. This range is defined relative to the dimensions of the original image, ensuring that the function focuses on moderately sized regions while ignoring areas that are too small or too large. For the

remaining polygons, their centroids (or focus points) are calculated using image moments. The moments provide the spatial distribution of the pixel intensities in a shape, allowing the function to compute the x and y coordinates of the centroid. Afterwards a bounding box is created around the approximated contour polygon.

For every valid segment, a dictionary is created to store its attributes. In summary, the `edge_detection` function methodically processes an image to isolate rectangular regions based on their contours and predefined size criteria. Through grayscale conversion, binary thresholding, morphological operations, and contour analysis, the function extracts meaningful geometric information while ensuring robustness and accuracy in its results.

3.2 Color correction

A uniform grey area in the image is required for color correction. The grey frame that surrounds the cube in the laboratory was used for this purpose. In order to recognise this reliably even with different resolutions and camera distances, a fixed limitation of the grey frame via the coordinates in the image is not possible. It should also be recognised under different lighting conditions, which is why color detection without prior color correction is ruled out. As color correction is not possible without these grey frames, this approach cannot be used either. This leaves object detection via edge detection. This was implemented in the `detect_outer_gray_frame` function. There, the input image is first smoothed and then filtered with a canny filter of the function `cv2.Canny`. This returns all edges in the input image. The threshold values for the canny filter were adjusted experimentally in order to achieve the best possible result. As it can still happen that the edges of the contours are not closed, these are closed using a morphological operation. The contours of the objects can now be found in the edge image using the command `cv2.findContours`. This returns a large number of contours, as all edges are recognised as contours. To get the contour of the grey frame, the contours are filtered. To do this, all contours are approximated by straight lines using the `cv2.approxPolyDP` function. As the frame is square, it can be recreated with four straight lines. Therefore, only the contours with four straight lines are checked. As the frame is the largest rectangular contour in the laboratory setting, the contour with four straight lines and the largest enclosed area must be the outer contour of the frame. A bounding box is placed around this contour, which encloses the object.

To compensate for inaccuracies in the detection, the bounding box is reduced by 5% around the centre point. To calculate the inner edge of the grey frame, a second bounding box is created, which in turn is 10% smaller than the reduced bounding box. The area between the two bounding boxes encloses the grey frame and serves as the grey area for the color correction.



Figure 10: image of cube with grey frame with drawn bounding boxes

The function `correctImage` implements the described proceed from Chapter 2.2. The Ground Truth values were determined in the image, we defined as ground truth. The color areas for subsequent color detection were also defined in this image. In our case, these the Ground Truth values were $\sigma_r = 18.23, \sigma_g = 18.92, \sigma_b = 18.84, \langle r \rangle = 145, \langle g \rangle = 155, \langle b \rangle = 173$. These values are hard-coded as parameters in the programme code. The values $\sigma'_r, \sigma'_g, \sigma'_b, \langle r' \rangle, \langle g' \rangle, \langle b' \rangle$ for the discolored input image are determined via the RGB values of the image within the two bounding boxes in the grey frame using the `getImageValues` function. The correction constants $\hat{c}_0, \hat{c}_r, \hat{c}_g, \hat{c}_b$ are then calculated. The correction constants are used to transform the RGB value of each pixel in the image in order to recalibrate it. The result is an image that resembles the coloring of the ground truth image and in which the colors of the surfaces can be detected later on, regardless of the actual lighting situation.



Figure 11: Original image before correction



Figure 12: Corrected image for color detection

3.3 Color detection

The color detection is located in the `color_filter` function. The thresholds for the HSV color space are also defined there. Each color is given a range in which the pixel values may lie in order to be assigned to a specific color. The thresholds are shown in Table 1. In OpenCV, the Hue value is defined in a range from 0 to 180°, the Saturation and Value values are defined in a range from 0 to 255. The color range perceived as red is in an angular range of 160° to 5° and therefore exceeds 0°. Two possible red color ranges are therefore defined.

Color	Lower H	Lower S	Lower V	Upper H	Upper S	Upper V
Red	0	90	70	5	255	255
	160	90	70	180	255	255
Orange	5	100	70	19	255	255
Yellow	20	50	100	44	255	255
Green	45	50	70	94	255	255
Blue	95	130	70	150	255	255
White	0	0	160	180	50	255

Table 1: HSV thresholds color filtering

When the function is executed, the RGB input image is first transformed into the HSV color space using the `cv2.cvtColor` function. The pixels retain their actual appearance, this is only represented by other values. A for loop is then run through, which calls up each possible color once. Firstly, a binary image is created from the input image in the for loop. A check is made for each pixel to see whether it lies within the value range of the current color. If this is the case, it is displayed as a white pixel in the binary image, otherwise it is black. The binary image is first opened and then closed with the `cv2.morphologyEx` function to close any holes in the areas and remove artefacts outside the relevant areas. This means that only cube surfaces that have this color should still be present as white surfaces.

This is followed by a search for contours using `cv2.findContours`. As there may still be artefacts that also have a contour, the contours must be filtered. The area of the contour, its squareness and the filling factor of a bounding box around the contour are decisive here. The area of the contour must correspond to between $\frac{1}{11}$ and $\frac{1}{8}$ of the area of the entire cube. As the cube has spaces between the colored areas, the threshold is designed to be slightly smaller at the bottom. The contour must have a ratio of width to height of at least 0.8 and a maximum of 1.2. This ensures that only approximately square objects are processed through their contour. Finally, the area of the recognised object within the contour must be greater than the product of the maximum extension in height and width of the contour. This corresponds to the filling factor and ensures that only rectangular areas are recognised.

If an object passes through this filter, it can be assumed that it is a colored area of the cube. The object is saved in a list and annotated with the corresponding color. The for loop then moves on to the next execution and searches for faces in the next color until all colors have

been processed. The function returns the list of all areas with color and position as well as a combined mask in which all colored areas are highlighted.

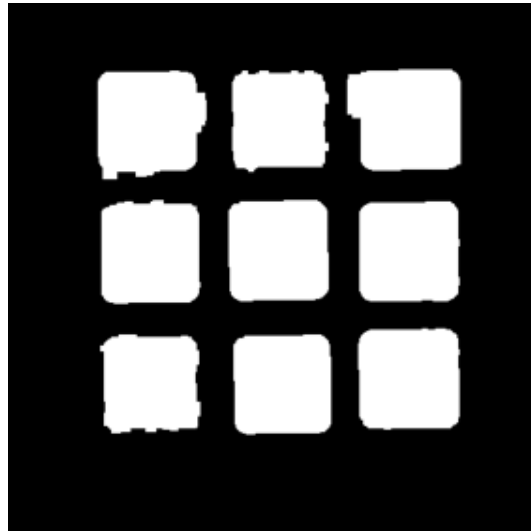


Figure 13: Mask of all nine detected colored surfaces

3.4 Merging

the two dictionaries show that different information can be obtained using the two detection methods. A general dictionary structure is used to handle the information correctly:

```
valid_element = {  
    "bbox",  
    "area_focus_point",  
    "color",  
    "grid_position",  
    "avg_hue"  
}
```

Not only the amount of information is different, but also the number of detected objects can differ. In order to make the collection of information redundant, the merging process is described in more detail below. This is further subdivided into the grouping of the x and y coordinates found, whereby the corresponding grid positions can be determined, and the actual merging of the detected segments. The Table 2 show an example of the color and edge detection dictionary that must be merged by the function.

bbox (X, Y, W, H)	area_focus_point (X, Y)	color	grid_position	avg_hue
291.0, 274.0, 83.0, 83.0	332, 315	Blau	None	113.1
81.0, 169.0, 81.0, 85.0	121, 211	Blau	None	113.8
80.0, 279.0, 82.0, 81.0	121, 319	Gelb	None	29.9
188.0, 63.0, 79.0, 83.0	227, 104	Gelb	None	29.9
185.0, 169.0, 82.0, 83.0	226, 210	Grün	None	69.6
189.0, 279.0, 80.0, 82.0	229, 320	Orange	None	8.8
292.0, 170.0, 82.0, 82.0	333, 211	Weiss	None	6.8
78.0, 63.0, 89.0, 89.0	122, 107	Weiss	None	12.7
282.0, 62.0, 93.0, 83.0	328, 103	Weiss	None	12.8

bbox (X, Y, W, H)	area_focus_point (X, Y)	color	grid_position	avg_hue
188.0, 279.0, 80.0, 81.0	228, 319	None	None	None
76.0, 281.0, 84.0, 78.0	119, 318	None	None	None
186.0, 170.0, 80.0, 80.0	225, 209	None	None	None
293.0, 160.0, 83.0, 90.0	334, 208	None	None	None
75.0, 63.0, 92.0, 91.0	118, 105	None	None	None
175.0, 62.0, 98.0, 84.0	223, 104	None	None	None
281.0, 63.0, 96.0, 85.0	329, 101	None	None	None

Table 2: Example dictionaries of color detection (left) and edge detection (right)

3.4.1 Grouping

Before the correct grid positions can be assigned, groups must be created to sort the entries found. This is achieved through the function `group_coordinates`, which clusters detected positions along the x and y axes using a dynamically determined threshold. This threshold is calculated based on the average bounding box size of detected objects, ensuring adaptable grouping.

The function takes the list of focus points as input and returns the mean of the group elements. It starts by placing the first value in an initial group. Then, it iterates through the rest of the values, adding each one to the current group if the difference between it and the previous value is within the threshold. If the gap is too large, a new group is created.

Once all values have been processed, the function checks if the number of groups matches the 3x3 structure. If the number of detected groups does not match the expected grid structure, the function returns `None`, indicating that proper grouping was not possible. The restriction of this method is, that at least one element in each row and in each column is present in the detection. The groups from the example dictionaries (Table 2) can be grouped as shown in the following table:

Typ	X-Groups			Y-Groups		
Color	121.3	227.3	331.0	104.7	210.7	318.0
Edge	118.5	225.3	331.5	103.3	208.5	318.5

Table 3: Grouping of focus points

3.4.2 Assign grid positions

Once grouped, each object is assigned a unique position in a 3x3 grid layout. This row and column indices are determined by finding the closest coordinate group in x and y direction. If the assignment process fails (due to improper grouping), the function returns unmodified entries.

3.4.3 Combining of dictionaries

Once a grid position has been assigned to each segment found, you can iterate through all entries and merge entries with the same grid positions. The following logic is applied

- If one detection method fails, the other is used exclusively.
- If both methods detect an object at the same position, their attributes (bounding box, area focus point, etc.) are merged using a weighted average. In this case, if the color detection is detecting something it is quite robust with minimal noise. In comparison the edge detection does not often detect a perfect square because of reflections or other noise therefore we applied the weighted average of `faktor_edge_dict = 1` and `faktor_color_dict = 2`
- The detection source is labeled as "color," "edge," or "both," depending on the contributions from each method.

Not only are the dictionaries merged, but also the individual groups. The function `merge_groups` ensures that the x and y coordinate groups are also combined using a weighted average approach. The resulting merged dictionary from Table 2: Example dictionaries of color detection (left) and edge detection (right) can be seen in the output panel in Figure 15

3.5 Compensation for missing segments

Even after merging, some grid positions may remain empty due to incomplete detection. The `interpolate_missing_entries` function compensates for these gaps by estimating missing elements based on geometry. This is done by the mean values of the merged x and y coordinate groups. These interpolated entries are labeled as "interpolated" to distinguish them from directly detected objects. The following Figure 14 shows the geometric relationship with which the missing segments are found. The blue dots represent the merged group coordinates with every x,y pair and the green dot the missing grid position. With this method missing segments can be compensated.

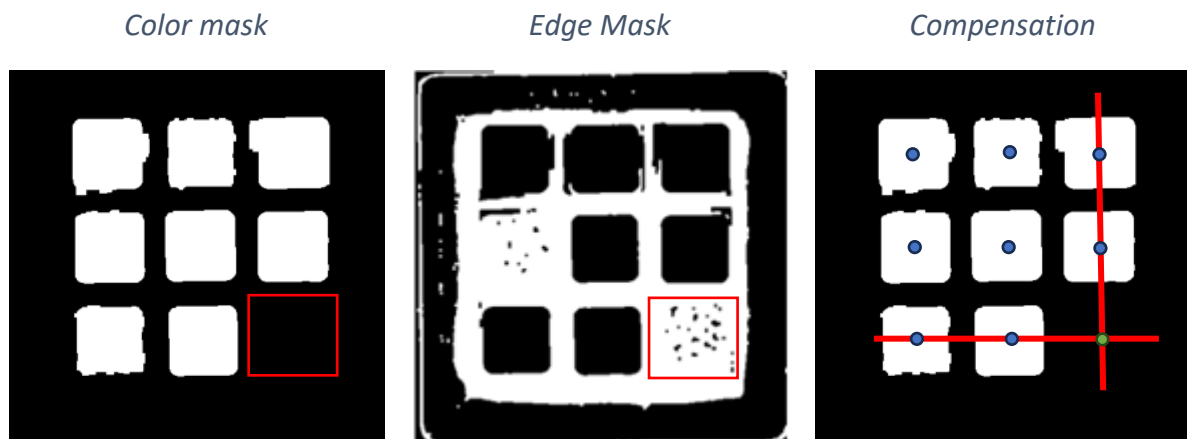


Figure 14: Compensation of missing segment

4 Test and Evaluation

To test and for evaluation of the algorithm and the individual components, two data sets of images were generated. Firstly, a set of images with the same lighting conditions but different colors and sides of the cube facing the camera. In the second set, the lighting conditions were changed so that brighter and darker images and images with different color influences were recorded.

A panel was created as the output of the program, which contains all the important information about the execution. Firstly, the largest element is the input image, which is located on the right-hand side. This image shows the outer contour and the two bounding boxes for defining the area for color correction. In addition, the individual areas are labelled with numbers at their centre. These numbers can be used to identify the individual areas in the table below the image. The color, the centre point and the type of detection are also noted in the table. A surface can be detected and identified either by edge detection, color detection or both. The values of the correction constants for color correction are shown in the table below.

There are three images on the left-hand side of the panel. At the top is the cropped, color-corrected image of the cube in which the color detection is performed. Below this is the mask from the edge detection, which attempts to recognise the relevant areas via their edges. Below this again is the combined mask from the color detection, which contains all objects whose pixels lie in one of the defined color areas.

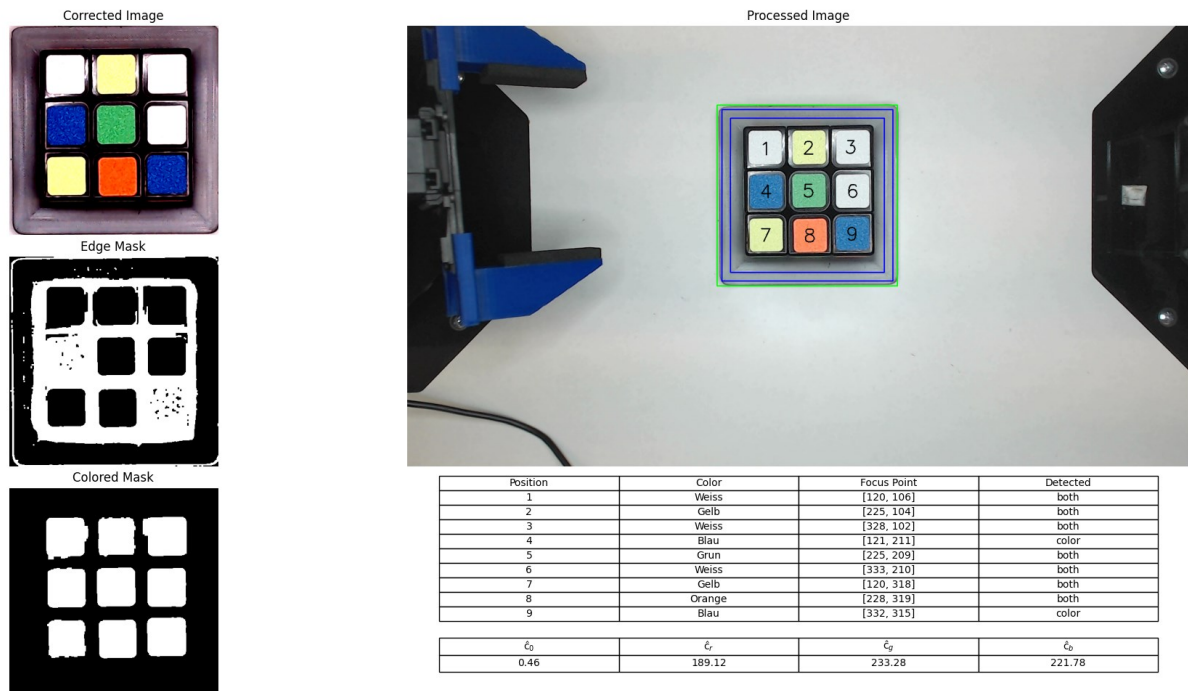


Figure 15: Debug and output panel

Testing the algorithm with different images showed that under neutral lighting, color recognition using the `color_filter` function reliably recognizes all six different colors of the nine fields. However, misclassifications can rarely occur with very similar colors. This is the case with red and orange, which lie directly next to each other on the HSV color wheel and are sometimes difficult to distinguish by the camera. Especially as the threshold values cannot be adapted to every image and its color appearance. In very bright light, overexposure can

occur, which makes the yellow on the cube appear very faint. The saturation and the value are very low, which can lead to confusion with white. A better threshold value cannot be determined here either, as white can assume a higher saturation and value in poorly exposed images. No distinction can be made using the hue value either, as this covers the entire color wheel for white.

If colors are not recognized by the color filter, this can be intercepted by edge detection. This works reliably for the colors green, yellow, white and orange. These colors are brighter colors and their brightness is certainly above the limit value for binarization of the image. With the colors red and blue, however, errors can occur with dark exposure, as these then appear very dark. In this case, the areas are not distinguished as such from the borders and no object can be detected.

In the event that a surface of the cube is neither detected by the color filter nor by the edge detection, compensation is used, which interpolates missing surfaces. This works as soon as at least one face in a respective row and column has been detected by the previous methods. This means that three detected tiles located on a diagonal are sufficient as minimum conditions to interpolate all missing surfaces. This at least makes it possible to determine the position of missing areas, and the color detection of these areas can then potentially be implemented using other methods.

5 Prospects

Possible improvements to the algorithm include robustness in relation to rotations. With the current implementation, the cube must be aligned as precisely as possible to the edges of the image. Slight rotations of $\pm 5^\circ$ can be compensated for in some cases but are not sufficiently robust to ensure reliable recognition. An idea here would be to make the detection of the grey frame robust against rotations and then rotate the image into a vertical orientation using the `cv2.minAreaRect` of the OpenCV library. The following implementation can then be retained.

The current implementation of the edge detection could also be enhanced by integrating an adaptive threshold method to create the black mask. For now, the threshold is hard codes and because of the color correction the results are acceptable but could be improved by methods like Otsu or the adaptive threshold method provided by OpenCV.

Furthermore, large color exposure influences cannot be compensated for with the implemented color correction. A more reliable approach here would be an implementation using deep learning. A convolutional neural network (CNN) can be trained accordingly with a large data set to reliably recognise and evaluate all colors under different lighting situations. To do this, the data set must contain all possible colors, brightnesses and illuminations to be robust against these influences. This approach could also be trained to be robust against rotation by further augmentation of the dataset.

Once reliable detection of the colored surfaces has been implemented, the robot arms of the laboratory workstation can be put into operation, initially to rotate the cube so that all sides can be analysed. A solution algorithm can then be implemented so that the robot arms can solve the magic cube independently.

6 Bibliography

- [1] „Contours,“ 15. 12. 2025. [Online]. Available: https://docs.opencv.org/3.4/d4/d73/tutorial_py_contours_begin.html.
- [2] „Canny Edge Detection,“ OpenCV, 28. 01. 2025. [Online]. Available: https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html.
- [3] D. Prof. Dr. rer. nat. Maier, Computer and Robot Vision Skript, Heilbronn, 2024.
- [4] „Open Source Computer Vision: OpenCV-Python Tutorials,“ 02 Dezember 2024. [Online]. Available: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html.

7 List of figures

Figure 1: Image of Rubik's Cube under lab conditions	1
Figure 2: Non-maximum suppression [2]	2
Figure 3: Color correction - Calibrated object [3] ($\sigma r = 11,7, \sigma g = 11,1, \sigma b = 11,2, r = 174,8, g = 173,8, b = 173,2$)	3
Figure 4: Color correction - Simulated data [1] ($\sigma' r = 9,3, \sigma' g = 8,9, \sigma' b = 9,0, r' = 163,4, g' = 138,7, b' = 138,2, cr = 30; cg = 0; cb = 0; c0 = 0,8$)	4
Figure 5: Color correction - re-calibrated image [1] ($\sigma' r = 11,6, \sigma' g = 11,1, \sigma' b = 11,2, r' = 174,4, g' = 173,2, b' = 172,8$)	5
Figure 6: Representation of Hue in HSV color space [3]	6
Figure 7: Process Diagram	7
Figure 8: Preprocessing of edge detection	8
Figure 9: Result of edge detection	8
Figure 10: image of cube with grey frame with drawn bounding boxes	10
Figure 11: Original image before correction	10
Figure 12: Corrected image for color detection	10
Figure 13: Mask of all nine detected colored surfaces	12
Figure 14: Compensation of missing segment	14
Figure 15: Debug and output panel	15

8 List of tables

Table 1: HSV thresholds color filtering	11
Table 2: Example dictionaries of color detection (left) and edge detection (right)	13
Table 3: Grouping of focus points	13